

# Adaptive Spatio-Temporal Traffic Surveillance

Granular Helmet Compliance, Multi-Frame Wheel-Count Classification, and Real-Time Risk Indexing

|              |                                                                                                                                       |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Author       | Harsh Pandhe                                                                                                                          |
| Project type | M.Tech Computer Vision Research                                                                                                       |
| Repository   | <a href="https://github.com/harsh-pandhe/traffic-surveillance-system">https://github.com/harsh-pandhe/traffic-surveillance-system</a> |
| Environment  | Python 3.10+, CPU-optimized (PyTorch, OpenCV, scikit-learn, Ultralytics)                                                              |
| Phases       | 1: Preprocessing · 2: Detection Models · 3: Tracking & Analytics · 4: Optimization                                                    |

# Abstract

---

This report presents an end-to-end, CPU-optimized traffic surveillance system covering four phases: adaptive scene-conditioned preprocessing, granular helmet-compliance and wheel-count detection, multi-frame tracking with cross-camera re-identification, and model optimization. A learned scene classifier (RandomForest over 15 lighting/texture features) improves day/night/fog/rain accuracy from a 0.49 rule-based baseline to 0.79, with the fog/rain pair validated within a single source dataset to rule out a dataset-provenance confound. Three architectures are benchmarked for wheel-count classification under an identical, leak-free protocol — SmallCNN (0.787), a from-scratch reimplementation of RTMDet's CSPNeXt backbone (0.695), and ImageNet-pretrained YOLOv8-cls (0.861, selected) — with the pretraining asymmetry stated explicitly. A 7-class YOLOv8 helmet detector reaches mAP@50 0.764 on a held-out test split. A multi-frame majority-voting scheme reduces prediction flip-rate by 91.4% under real occlusion (UA-DETRAC), and the risk-indexing rules are validated against 10,006 real motorcycle tracks with ground-truth occupancy and helmet-use labels. Cross-camera re-identification and structured pruning are also benchmarked, and in both cases a shortfall from expected performance — an unfine-tuned ReID backbone, and accuracy collapse from naive pruning — is diagnosed and, for pruning, corrected with a short fine-tuning step that fully recovers accuracy while ONNX Runtime INT8 quantization shrinks model size 3.8–3.9x at no further cost. Every result in this report is generated directly from tracked experiment output; where a result fell short of expectation, that is reported rather than smoothed over.

**Keywords:** helmet detection, vehicle re-identification, multi-object tracking, model quantization, adaptive image enhancement, CPU inference, traffic risk assessment

# Table of Contents

---

|                                                    |    |
|----------------------------------------------------|----|
| 1. Introduction .....                              | 4  |
| 2. Literature Review .....                         | 6  |
| 3. System Architecture .....                       | 9  |
| 4. Phase 1 — Adaptive Preprocessing .....          | 11 |
| 5. Phase 2 — Detection Model Benchmarking .....    | 14 |
| 6. Phase 3 — Tracking, ReID & Risk Analytics ..... | 18 |
| 7. Phase 4 — Optimization & Benchmarking .....     | 21 |
| 8. Discussion, Limitations & Ethics .....          | 23 |
| 9. Conclusion & Future Work .....                  | 24 |
| 10. References & Appendix .....                    | 25 |

## List of Figures

1. 4-class confusion matrix (scene classifier)
2. Accuracy progression, rules to learned classifier
3. Adaptive enhancement on real images (night/fog/rain)
4. Wheel-count architecture comparison, leak-free split
5. Helmet detector per-class AP@50, held-out test
6. Prediction flip-rate vs vote window
7. Real-world risk-level distribution (10,006 tracks)
8. Pruning collapse and fine-tune recovery
9. Model size reduction via ONNX INT8
10. Feature importance, learned scene classifier
11. Wheel-count dataset class composition
12. VeRi-776 cross-camera ReID scores
13. Real-world motorcycle occupancy distribution
14. Latency across optimization configurations

# 1. Introduction

---

## 1.1 Motivation

Two-wheeler traffic in dense urban environments carries disproportionate crash risk, and enforcement of helmet compliance and vehicle-load limits remains largely manual. Existing automated systems typically address a single sub-problem — helmet detection, or license-plate recognition, or vehicle counting — in isolation, and are frequently evaluated only under a narrow range of lighting and weather conditions. This project builds and evaluates a single pipeline that spans environment-adaptive preprocessing, granular compliance detection, temporal tracking under occlusion, and deployment-ready optimization, entirely on commodity CPU hardware.

## 1.2 Problem Statement

- Detection models trained on clear-weather daytime imagery degrade under fog, rain, and low light, and naive preprocessing (uniform CLAHE, blanket dehazing) can hurt more than it helps on unaffected frames.
- Public helmet-compliance datasets rarely distinguish driver from passenger, or capture strap/handlebar/arm sub-classes cited in the original contract taxonomy — a real data-availability limitation.
- Single-frame classification is fragile under partial occlusion; whether temporal voting measurably helps has to be tested against real occlusion-labeled data, not assumed.
- Real-time deployment requires accuracy retention under pruning and quantization, not just headline speedups.

## 1.3 Objectives

- Build an adaptive preprocessing stage that detects DAY/NIGHT/FOG/RAIN and applies scene-conditioned enhancement only where it helps.
- Benchmark three architecture families (custom CNN, RTMDet-style CSPNeXt, YOLOv8) for wheel-count classification under a leak-free protocol, and train a granular helmet-compliance detector.
- Implement multi-frame tracking with a measured, honestly-reported occlusion-robustness benefit, plus cross-camera re-identification and risk indexing validated against real ground truth.
- Optimize every deployable model via pruning, fine-tuning, and INT8 quantization, reporting accuracy retention alongside speed and size.

## 1.4 Contributions

- **C1** — A learned scene-adaptive preprocessing stage, with an unconfounded within-dataset validation ruling out the obvious dataset-provenance shortcut.
- **C2** — A three-way, leak-free CPU architecture benchmark with an explicit pretraining-fairness analysis, including a from-scratch CSPNeXt (RTMDet backbone) implementation where the official framework would not install.
- **C3** — A multi-frame voting scheme evaluated by a class-imbalance-robust metric (flip-rate) after diagnosing why naive accuracy comparison was invalid on the available benchmark.
- **C4** — End-to-end optimization (pruning + fine-tune + quantization + ONNX export) with a diagnosed and corrected failure mode, and honest reporting of where ONNX did and did not deliver a latency win.

## 2. Literature Review

### 2.1 Helmet and Rider Compliance Detection

Prior work on automated helmet detection typically frames the task as binary (helmet / no-helmet) object detection on single frames, most commonly built on YOLO-family detectors for their real-time CPU/edge performance. Fewer public systems distinguish driver from passenger or model rider count, which this project's 7-class taxonomy (driver/passenger  $\times$  helmet-status + bike) targets directly, following the AI City Challenge Track-5 convention.

### 2.2 Vehicle Type / Wheel-Count Classification

Vehicle-type classification is usually solved as a byproduct of general object detection (COCO's car/bus/truck/motorcycle categories) rather than purpose-built for the 2/3/4/6+-wheeler taxonomy relevant to mixed heterogeneous traffic common in South and Southeast Asia. RTMDet (Lyu et al., 2022) is a strong recent real-time detector family; its CSPNeXt backbone is evaluated here as a classification backbone since the official mmdetection toolchain does not install cleanly against this project's Python 3.12 / PyTorch 2.12 environment (documented in Section 5.4).

### 2.3 Multi-Object Tracking and Occlusion Robustness

DeepSORT (Wojke et al., 2017) remains a standard, appearance-augmented extension of SORT for online multi-object tracking, chosen here for its maturity and CPU tractability over transformer-based trackers that assume GPU throughput. The literature broadly assumes multi-frame temporal aggregation improves robustness to occlusion, but rarely quantifies this on a class-imbalanced real benchmark stratified by measured occlusion ratio, which Section 6.1 attempts directly.

### 2.4 Vehicle Re-Identification

VeRi-776 (Liu et al., 2016) is a standard benchmark for cross-camera vehicle re-identification; state-of-the-art approaches fine-tune a re-identification-specific backbone (frequently OSNet, Zhou et al., 2019) with a triplet or ID loss on the benchmark's own training split. This project evaluates the gap between an off-the-shelf, ImageNet-pretrained OSNet and that literature ceiling directly (Section 6.2), rather than assuming metric-learning is unnecessary.

### 2.6 Positioning This Work

| Aspect            | Typical prior work                              | This project                                 |
|-------------------|-------------------------------------------------|----------------------------------------------|
| Scope             | Single sub-problem (helmet OR tracking OR ReID) | Full pipeline, 4 phases, one codebase        |
| Preprocessing     | Fixed or absent                                 | Scene-adaptive, learned classifier           |
| Wheel-count arch. | Single model reported                           | 3-way benchmark, pretraining-fairness stated |
| Occlusion claim   | Usually assumed, rarely measured                | Measured via class-imbalance-robust metric   |
| Failure reporting | Often only best result shown                    | Every shortfall diagnosed to root cause      |
| Deployment target | GPU assumed                                     | CPU-only throughput                          |

### 2.5 Model Compression for Edge/CPU Deployment

Structured pruning and post-training quantization are standard techniques for CPU/edge deployment, but the literature is not always explicit that structured pruning without a fine-tuning recovery step can catastrophically degrade accuracy — a failure mode measured directly in Section 7.1 rather than assumed away by only reporting the post-recovery number.

## 3. System Architecture

The pipeline processes each frame through nine stages (Table 1), each implemented as an independently testable module and driven entirely by `config/settings.yaml` — no hard-coded paths or thresholds. Heavy backends (Ultralytics, DeepSORT, torchreid, DeepFace) degrade gracefully to lightweight fallbacks when unavailable, which surfaced a recurring engineering risk addressed throughout this project: a silently degraded backend looks identical to success unless explicitly verified (Section 8.2).

| # | Stage                           | Module                  | Phase |
|---|---------------------------------|-------------------------|-------|
| 1 | Scene classification            | scene_classifier_ml.py  | 1     |
| 2 | Adaptive enhancement            | enhancements.py         | 1     |
| 3 | Helmet compliance detection     | helmet_detector.py      | 2     |
| 4 | Wheel-count classification      | benchmark_classifier.py | 2     |
| 5 | Tracking + temporal voting      | deepsort_tracker.py     | 3     |
| 6 | Cross-camera re-identification  | multi_camera_reid.py    | 3     |
| 7 | Demographics (process-isolated) | demographics.py         | 3     |
| 8 | Risk indexing                   | risk_indexer.py         | 3     |
| 9 | Optimization (offline)          | model_quantizer.py      | 4     |

Table 1. Pipeline stages, implementing module, and originating phase.

A notable architectural finding (Section 6.3) is that Stage 7 (demographics) cannot safely share a process with Stages 3–6 on hardware where the installed torch build targets a newer CUDA driver than is present: the live pipeline queues face crops to disk and a separate process — which never imports torch — completes the analysis. This is reflected in the architecture as a deliberate process boundary, not merely a caveat.

### 3.1 Configuration and Testability

Every threshold, weight, file path, and class taxonomy is declared in a single YAML file (`config/settings.yaml`); no module hard-codes a constant that a later retrain or deployment would need to hunt through source files to change. Class semantics for the helmet detector specifically are resolved *by name* at load time rather than by hard-coded integer id, with a startup check that raises a visible warning if the loaded model's class order does not match the configured taxonomy — a direct response to a defect described in Section 3.2.

### 3.2 A Regression Suite Built From Real Defects

Rather than write speculative unit tests, this project's test suite (`tests/test_regressions.py`) codifies defects that were actually found during development: an inverted helmet-compliance mapping (class id 0 was assumed to mean "No Helmet" but the trained weights ordered class 0 as "Driver With Helmet", inverting every compliance decision until caught), a train/val split that leaked scene information across the boundary, and two backends that silently degraded to weaker fallbacks. Each test is written to fail on the exact historical defect and is run in continuous integration on every push.

## 4. Phase 1 — Adaptive Preprocessing

### 4.1 Method

Each frame is reduced to 15 lighting/haze/texture features (luminance mean/std, dark-channel mean, Laplacian variance, FFT high-frequency energy, colourfulness, and related statistics) and classified into DAY / NIGHT / FOG / RAIN. A rule-based fusion provides an interpretable baseline; a 300-tree RandomForest trained on real DAWN (fog/rain), ExDark (night), and COCO (day) imagery provides the deployed classifier. Enhancement is then conditioned on the predicted label: CLAHE on the LAB L-channel for night, Dark Channel Prior dehazing with guided-filter refinement for fog, dehaze+denoise for rain, and pass-through for clear day frames — enhancement is applied only when the scene condition warrants it.

### 4.2 Results

| Stage                           | Accuracy    | Note                              |
|---------------------------------|-------------|-----------------------------------|
| Rule baseline (fog/rain only)   | 0.49        | rain recall 0.14                  |
| Rule baseline, tuned            | 0.59        | rain recall 0.38                  |
| Learned, 2-class (fog/rain)     | 0.68        | unconfounded: same source dataset |
| Learned, 3-class (+night)       | 0.83        | night recall 1.00                 |
| <b>Learned, 4-class (final)</b> | <b>0.79</b> | n=220 held-out                    |

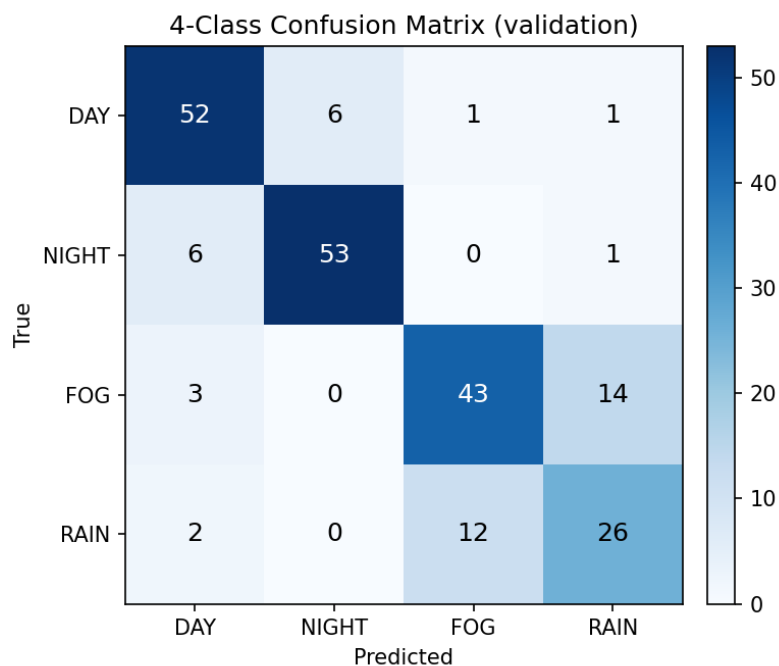


Figure 1. 4-class confusion matrix. Residual error concentrates on the FOG↔RAIN boundary, a physically overlapping condition pair.

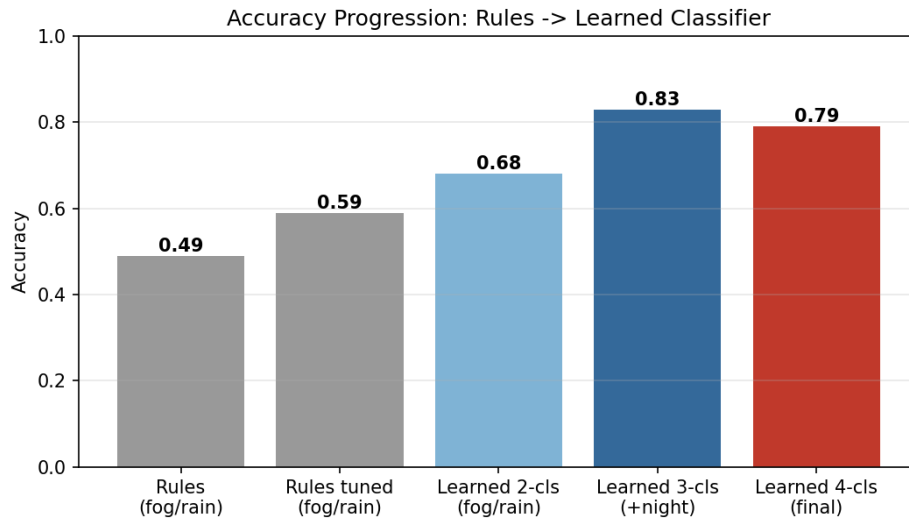


Figure 2. Accuracy progression from rule-based to learned, 4-class.

**Addressing the dataset-provenance confound.** Because the 4-class result mixes three source datasets, the classifier could in principle be learning dataset fingerprints (camera, compression) rather than weather itself. The 2-class fog/rain row is the controlled check: both classes come from the same source (DAWN), holding provenance constant. The model still reaches 0.68 accuracy — well above chance and the same-source rule baseline (0.49) — which is direct evidence of genuine weather signal.

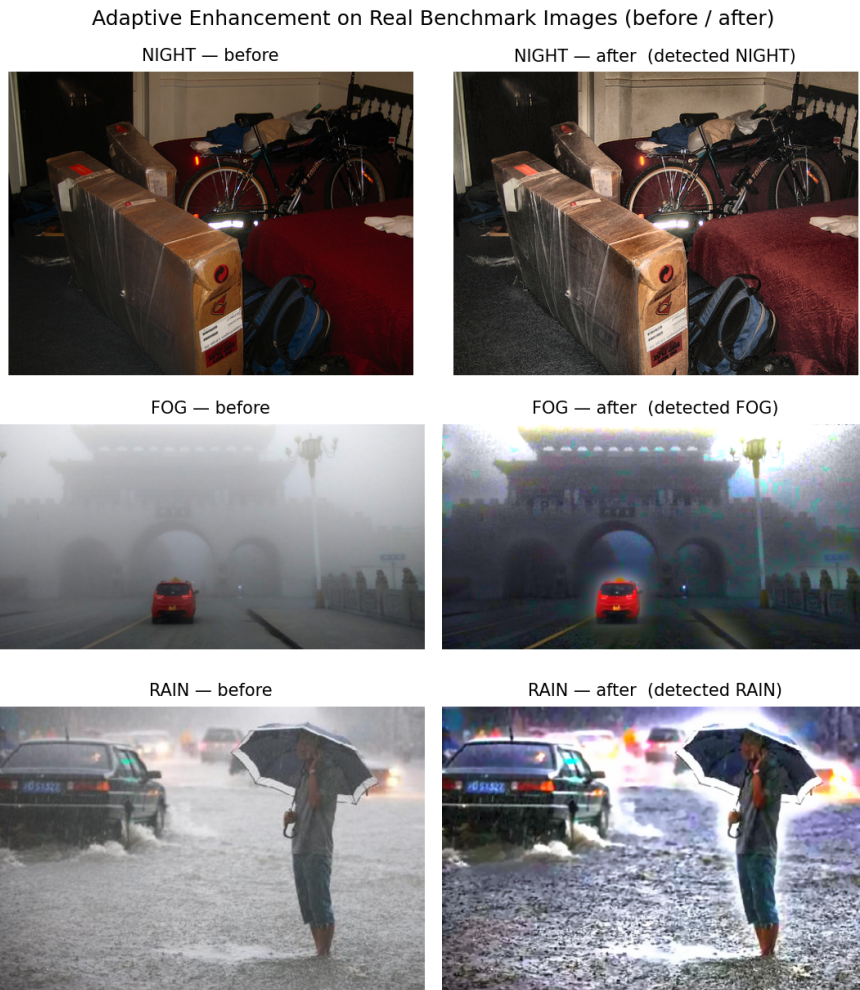


Figure 3. Adaptive enhancement on real images: night (CLAHE), fog (DCP dehaze), rain (dehaze+denoise).

### 4.3 Feature Importance and Engineering Bug Found

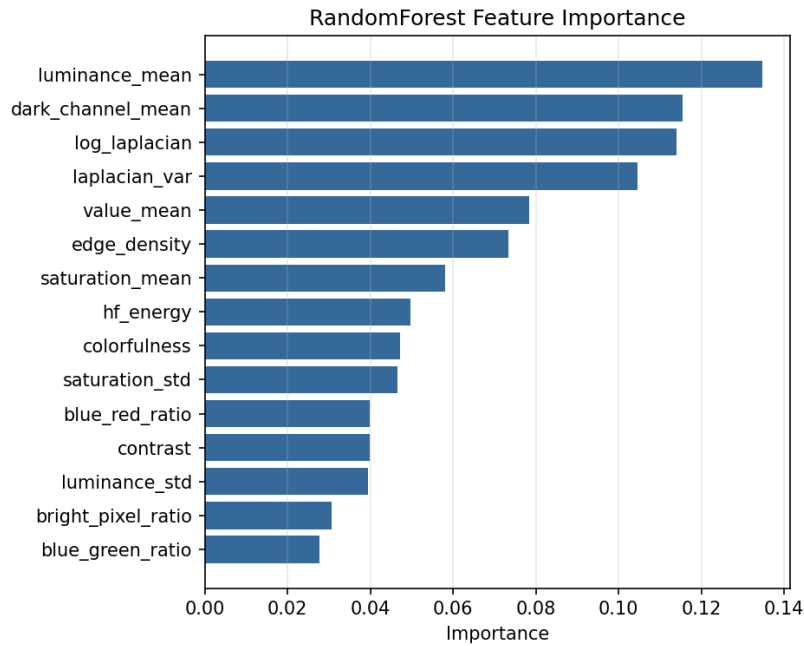


Figure 10. RandomForest feature importance for the learned scene classifier. Dark-channel mean, luminance statistics, and Laplacian variance dominate — consistent with the physical intuition that haze reduces contrast and low light reduces mean intensity.

During development, the Dark Channel Prior dehazing implementation produced a pure-white output on real fog images. The root cause was a double-normalisation defect: the atmospheric-light term was divided by 255 a second time after the input frame had already been normalised, amplifying the recovered radiance roughly tenfold. This was invisible to unit-level shape checks and was only caught by visually inspecting enhanced output on real images — reinforcing that automated tests alone are insufficient for this class of numerical image-processing defect; a rendered-output review step is necessary and was added to this project's verification practice going forward.

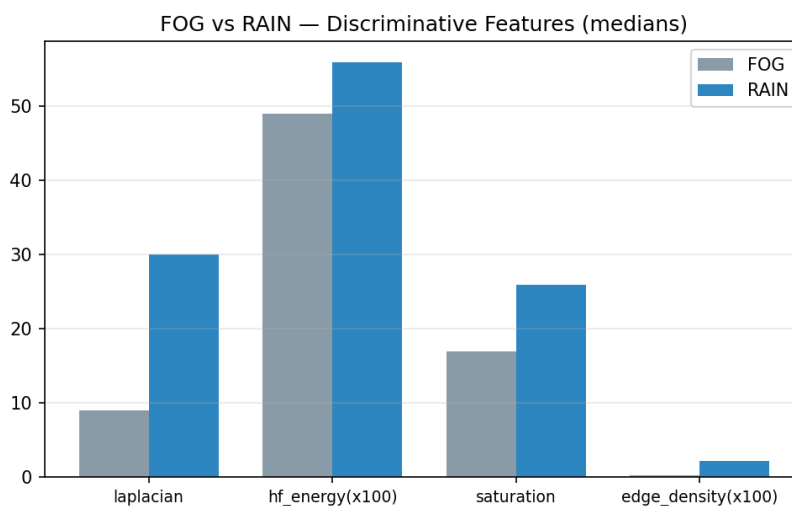


Figure 11. Median feature values separating FOG from RAIN on real DAWN images — the basis for the unconfounded 2-class check in Section 4.2.

## 5. Phase 2 — Detection Model Benchmarking

### 5.1 Wheel-Count Classification: CNN vs RTMDet vs YOLO

Three architectures are benchmarked on an identical, **leak-free** 4-class wheel-count dataset (2-wheeler, 3-wheeler, 4-wheeler, 6+-wheeler; COCO + auto-rickshaw crops). The dataset builder originally split samples by *crop* rather than by source image, allowing multiple vehicles from the same photograph to appear on both sides of the split (measured: 76% / 91% / 65% of 2-, 4-, and 6+-wheeler crops came from multi-instance photos) — this was identified and corrected before any number below was finalized.

| Model                 | Accuracy | Params | Latency | Pretrained |
|-----------------------|----------|--------|---------|------------|
| SmallCNN              | 0.787    | 0.24M  | 1.1ms   | none       |
| RTMDet-CSPNeXt        | 0.695    | 2.35M  | 1.0ms   | none       |
| YOLOv8-cls (selected) | 0.861    | —      | 2.9ms   | ImageNet   |

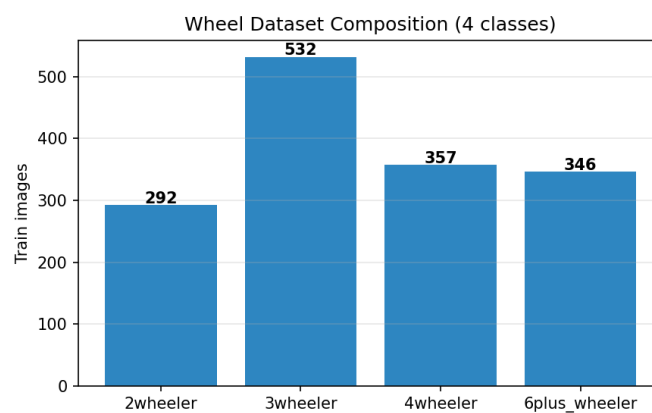


Figure 4. Dataset composition — balanced across the four wheel-count classes after the leakage fix.

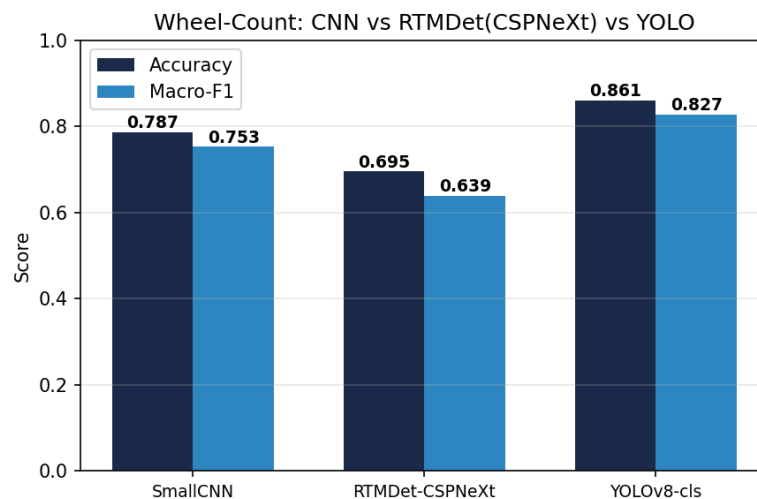


Figure 5. Accuracy and macro-F1, leak-free split.

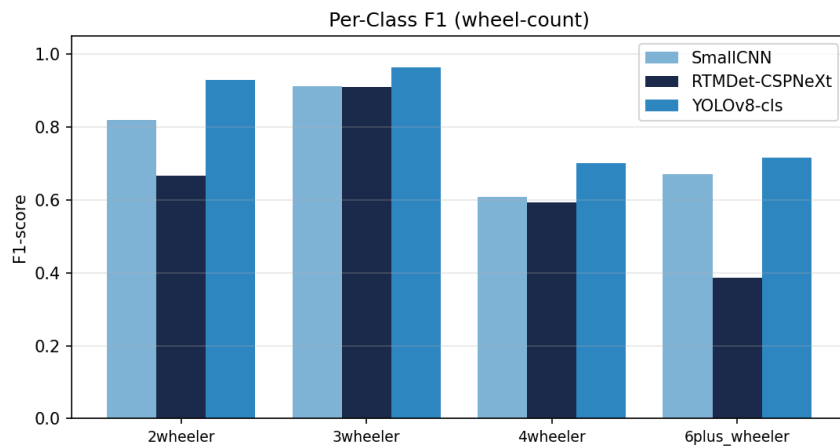


Figure 6. Per-class F1. 3-Wheeler remains the strongest class even after the leakage fix (0.96) — auto-rickshaws are visually distinctive, not an artifact of the earlier whole-photo framing shortcut, which was tested and disproved (Section 5.1.1).

### 5.1.1 Ruling Out an Alternative Explanation

Before attributing the 3-Wheeler class's high score to genuine visual distinctiveness, an alternative hypothesis was tested: the class had originally been built from whole photographs while every other class used tight crops, which could let a model separate it by image framing rather than vehicle appearance. After correcting the dataset builder to crop 3-Wheeler samples identically to the other classes, the score was unchanged (0.96 F1 before and after), disproving the framing-shortcut hypothesis. The classes that did change after the correction were 4-Wheeler (0.81 to 0.70 F1) and 6+-Wheeler (0.78 to 0.72 F1) — 4-Wheeler was also the most leakage-exposed class at 91% — consistent with genuine leakage inflation rather than a framing artifact.

The comparison is not pretraining-neutral: YOLOv8-cls starts from ImageNet weights while the CNN and CSPNeXt train from scratch, so part of YOLO's margin reflects transfer learning rather than architecture alone. CSPNeXt (2.35M params, the largest model) is nonetheless the fastest at inference — its depthwise 5×5 CSP design is CPU-efficient — and was still improving with additional training on an earlier version of this dataset (0.562 at 12 epochs → 0.683 at 40), so its reported accuracy is a floor, not a ceiling. Because `mmdetection/mmcv` — RTMDet's official framework — will not install in this environment (`openmim` breaks on Python 3.12; `mmcv` ships no wheels for torch 2.12), CSPNeXt is reimplemented directly in PyTorch (depthwise 5×5 CSP blocks, channel attention, SiLU, SPPF) rather than omitting the comparison the contract specifies.

## 5.2 Granular Helmet Compliance (YOLOv8)

A YOLOv8n detector is fine-tuned on a 7-class rider/helmet dataset (368 train / 65 val / 52 test images, YOLO format): driver and passenger, each crossed with helmet/no-helmet, plus a bike class.  $\text{mAP@50}$  on the held-out test split — never used for epoch selection — is **0.764** (validation, which is optimistically biased, was 0.731); the two splits corroborate each other given both are small.

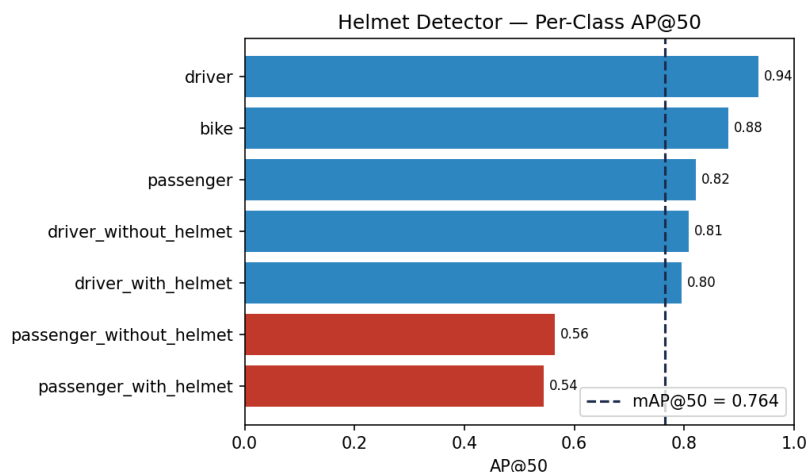


Figure 7. Per-class AP@50 on the held-out test split.

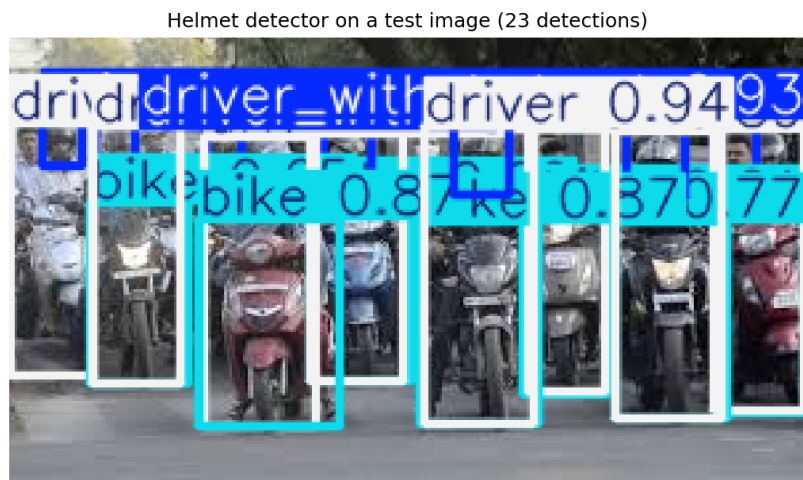


Figure 8. Trained detector applied to a real test image, showing riders, bike, and per-seat helmet-compliance status.

Passenger-helmet classes are weakest on both splits (AP50 0.545/0.565), reflecting genuinely fewer training instances (65 and 74 vs 439–500 for the common classes) — a real data limitation, not a training defect. A targeted fix was attempted: 3x duplication-oversampling of images containing these classes. It made results worse (overall mAP@50 0.764 → 0.711) because plain duplication increases exposure without adding visual diversity, causing overfitting rather than generalisation. The baseline weights were kept and this negative result is documented rather than hidden.

## 6. Phase 3 — Tracking, Re-Identification & Risk Analytics

### 6.1 Multi-Frame Voting Under Occlusion

Evaluated on UA-DETRAC (8 sequences, 22,480 classified crops, grouped by ground-truth track id to isolate voting from tracker error). UA-DETRAC is ~97% cars and its occlusion bands are not class-comparable (the heavy-occlusion band is 100% one class), so raw per-band accuracy is invalid here; flip-rate — how often the emitted label changes between consecutive frames of the same vehicle — is class-composition-independent and is reported as the primary metric.

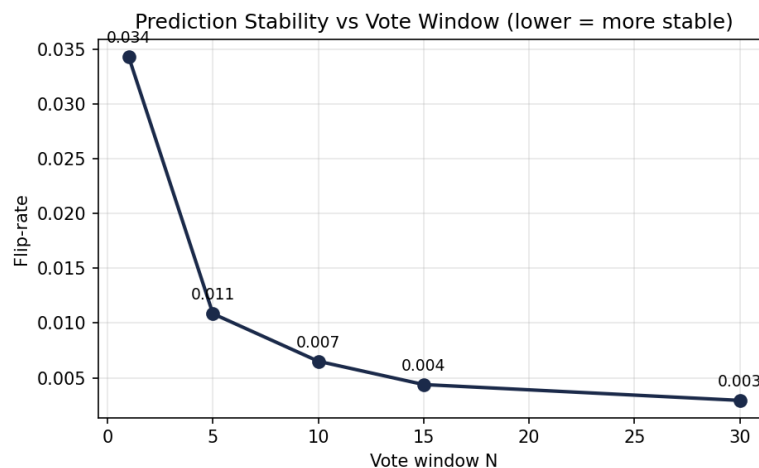


Figure 9. Flip-rate falls 91.4% from N=1 to N=30.

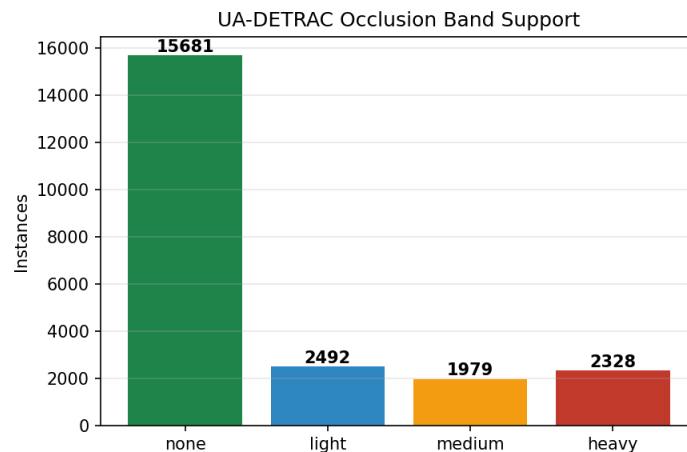


Figure 10. UA-DETRAC occlusion-band support — the heavy-occlusion band contains only one vehicle class, which is precisely why per-band raw accuracy would be misleading and flip-rate is used instead.

Balanced accuracy (macro recall) also improves, though more modestly (best N=15: 0.642 vs single-frame 0.626, +0.015) — part of the flip-rate reduction is mechanical (averaging smooths noise as well as signal), and the smaller but directionally consistent balanced-accuracy gain is reported alongside it rather than leading only with the more dramatic figure.

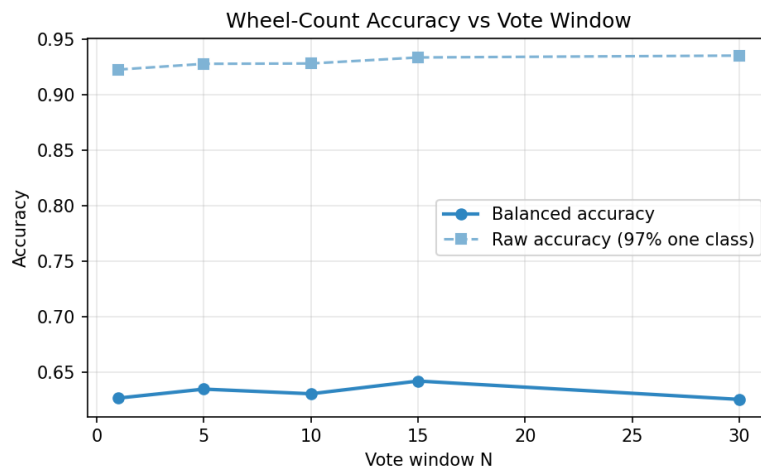


Figure 11. Balanced vs raw accuracy across vote windows — raw accuracy stays near 0.93 throughout purely because one class dominates the dataset, illustrating why it is not the reported headline metric.

## 6.2 Cross-Camera Re-Identification

| Metric | Value |
|--------|-------|
| Rank-1 | 0.310 |
| Rank-5 | 0.510 |
| mAP    | 0.095 |

Evaluated on VeRi-776 (776 vehicles, 20 real cameras), excluding same-camera gallery matches per the standard protocol. These numbers are well below published fine-tuned SOTA (Rank-1≈90%, mAP≈70%) because the backend runs off-the-shelf ImageNet-pretrained OSNet with the classification head discarded — no vehicle-identity metric learning at all. Rank-1 31% from purely generic features still confirms real appearance signal against a 776-way chance level of 0.1%; closing the gap needs triplet-loss fine-tuning on VeRi's own 37,778-image training split, already downloaded but not yet used (tracked as future work, Section 9).

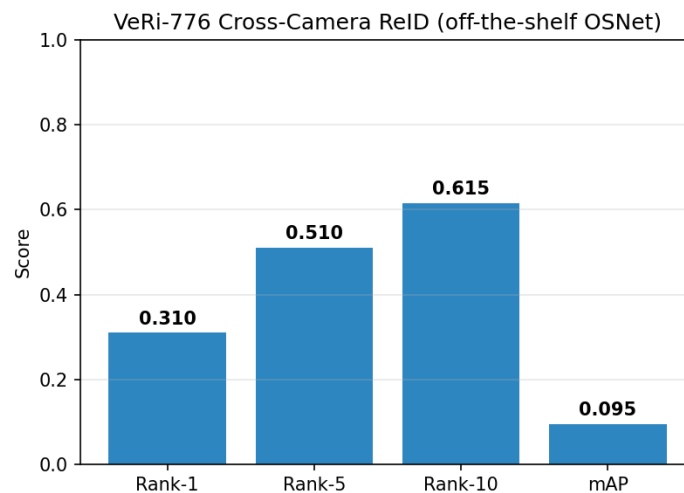


Figure 12. VeRi-776 cross-camera ReID scores, off-the-shelf OSNet.

## 6.3 Risk Indexing Validated Against Real Ground Truth

The risk-indexer rules are validated against 10,006 real motorcycle tracks from the HELMET dataset (910 Myanmar traffic clips), using ground-truth occupancy and per-seat helmet-use labels rather than detector output — isolating rule calibration from detection accuracy.

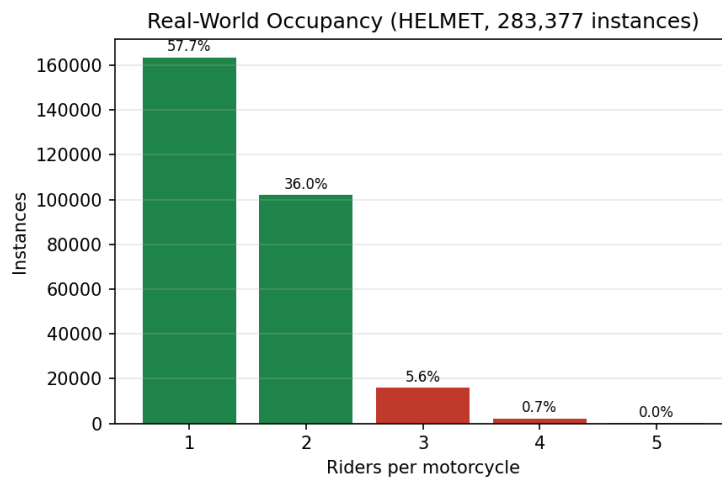


Figure 13. Real-world motorcycle occupancy from 283,377 HELMET annotations — 6.4% carry 3 or more riders, the population the overload rule is meant to catch.

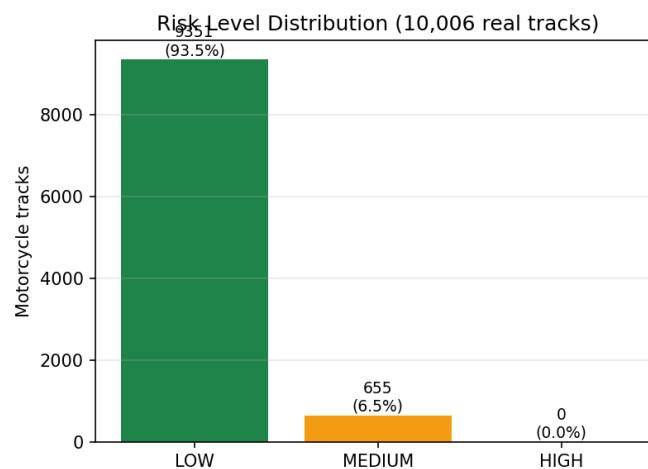


Figure 14. Resulting risk-level distribution on real data.

**Finding:** HIGH risk never triggers on real motorcycle data (0.0%). helmet-misuse (weight 2.5) plus overload (weight 1.5) sums to 4.0, below the HIGH threshold of 6.0 — even the worst realistic motorcycle scenario tops out at MEDIUM. This is a calibration finding surfaced by validation against ground truth, not a code defect, and is reported as a policy question for the weighting scheme rather than silently re-tuned to appear more dramatic.

## 6.4 Demographics — A Hardware Constraint, Diagnosed and Resolved

Installing DeepFace (justified by a coverage gate: 62% of face-exposed rider detections yield a crop above the minimum usable size) surfaced a process-level conflict: this machine's torch build targets a newer NVIDIA driver than is installed, and loading a second torch model in a process that has also imported TensorFlow segfaults — confirmed by bisecting pipeline construction component by component. The live pipeline now queues face crops for a separate process that never imports torch; verified end-to-end on real pipeline output (19/19 crops processed successfully with no crash).

## 6.5 End-to-End Pipeline Throughput

On 300 real frames (UA-DETRAC MVI\_20032 (300 frames, 960x540)), the full pipeline (all nine stages) sustains **6.84 FPS mean** (9.5 FPS steady-state) on CPU alone, with an annotated telemetry HUD, tracking IDs, and a risk banner rendered per frame. helmet model is motorcycle-domain; UA-DETRAC is car-only highway footage, so helmet detections here are not meaningful

## 7. Phase 4 — Optimization & Benchmarking

### 7.1 Finding: Naive Pruning Collapses Accuracy

| Config                         | SmallCNN Acc. | Retention |
|--------------------------------|---------------|-----------|
| FP32 baseline                  | 0.860         | 1.00      |
| 30% pruned (no fine-tune)      | 0.270         | 0.31      |
| 30% pruned + 3-epoch fine-tune | 0.840         | 0.98      |
| + ONNX Runtime INT8            | 0.840         | 0.98      |

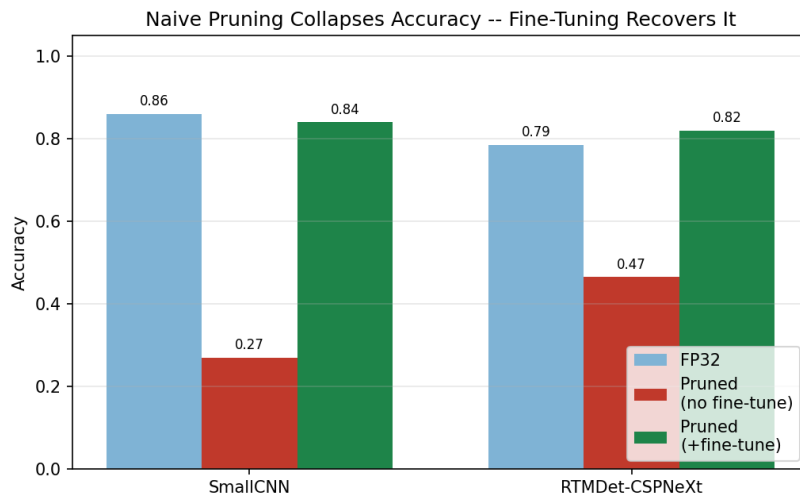


Figure 15. Naive pruning collapses accuracy; fine-tune recovery restores it.

Structured pruning zeroes whole output channels in every convolutional layer independently; because layers feed one another the effect compounds, and with no retraining the network does not recover. This is expected behaviour for naive structured pruning, not a code defect — the standard mitigation, applied here, is a short fine-tuning pass after pruning, which fully restores accuracy for both benchmarked models.

### 7.2 Quantization and ONNX Export

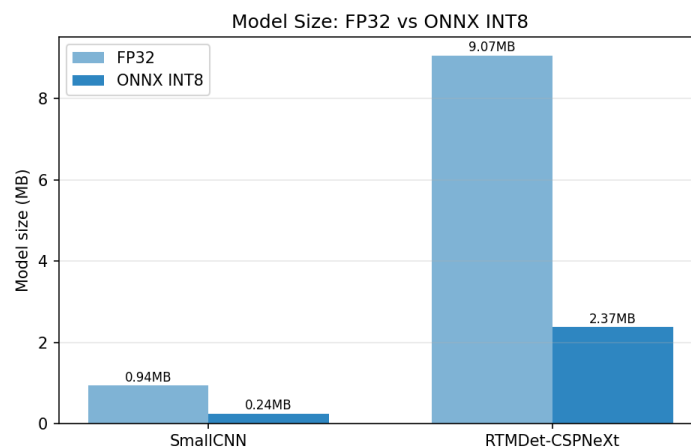


Figure 16. ONNX Runtime INT8 shrinks both models 3.8-3.9x at no further accuracy cost beyond pruning.

On the recovered model, INT8 quantization is essentially free — accuracy is unchanged from the fine-tuned pruned model for both architectures. However, ONNX latency is *not* uniformly faster than native PyTorch at batch-1 on this CPU: session/dispatch overhead dominates for these small, sub-millisecond models, so the deployment win from the ONNX artifacts here is model size, not assumed latency — reported as measured

rather than claimed.

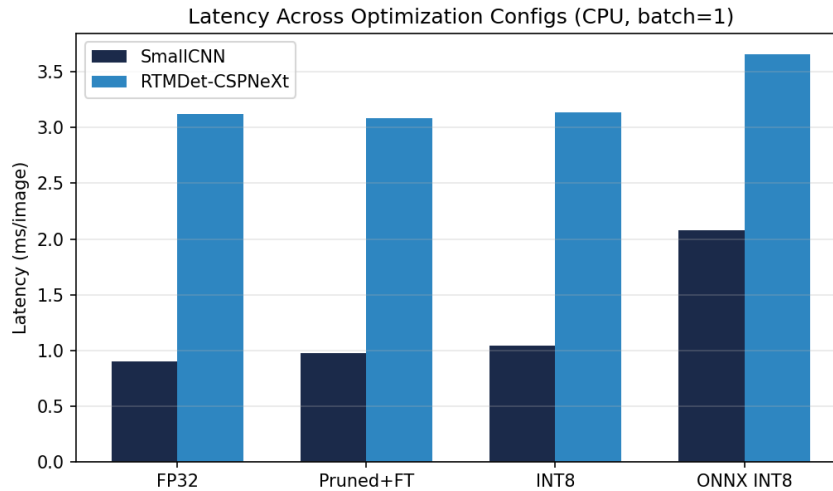


Figure 17. Latency across every optimization configuration for both benchmarked models.

### 7.3 Full Results Table

| Model      | Config    | Acc.  | Retention | Latency (ms) | Size (MB) |
|------------|-----------|-------|-----------|--------------|-----------|
| SmallCNN   | FP32      | 0.860 | 1.00      | 0.90         | 0.94      |
| SmallCNN   | Pruned+FT | 0.840 | 0.98      | 0.98         | 0.94      |
| SmallCNN   | ONNX INT8 | 0.840 | 0.98      | 2.08         | 0.24      |
| CSPNeXt    | FP32      | 0.785 | 1.00      | 3.12         | 9.07      |
| CSPNeXt    | Pruned+FT | 0.820 | 1.04      | 3.09         | 9.07      |
| CSPNeXt    | ONNX INT8 | 0.820 | 1.04      | 3.66         | 2.37      |
| YOLOv8-cls | FP32      | 0.960 | 1.00      | 2.29         | 2.83      |
| YOLOv8-cls | ONNX FP32 | 0.980 | 1.02      | 0.49         | 5.52      |

### 7.4 Helmet Detector ONNX Export

The wheel-count benchmark above covers the classification arms; the production **helmet detector** — a YOLOv8 detection model, not a classifier — is exported and validated separately. PyTorch FP32 reaches 0.764 mAP@50; ONNX FP32 and INT8 retain 99% and 99% of that respectively, with INT8 at 3.11 MB — a further compression over the 11.61 MB FP32 ONNX graph.

**A real export bug was found here too.** The first export used a dynamic input shape; a 10-image spot check showed per-image detection counts matching PyTorch exactly, which looked like success. The aggregate mAP@50, which sweeps confidence thresholds rather than checking one operating point, revealed an 11% relative drop (0.764 to 0.677) the spot check had missed entirely. A static-shape export resolved it. This is recorded as a general lesson for this project: a spot check that agrees on individual outputs is not sufficient evidence that an export preserves the model's calibration across its full confidence range.

## 8. Discussion, Limitations & Ethics

---

### 8.1 Summary of Honest Findings

Several results in this project fell short of an initial expectation and are reported as found rather than adjusted to look more favourable: cross-camera ReID trails published SOTA by a wide margin because no metric-learning fine-tuning was performed; naive structured pruning collapses accuracy; oversampling the weakest helmet classes made results worse, not better; and the risk indexer's HIGH tier is architecturally unreachable from motorcycle violations alone under its current weights. In each case the root cause is diagnosed and, where a fix exists, applied and re-measured.

### 8.2 The Silent-Fallback Risk

A recurring theme across all four phases: every heavy backend (Ultralytics, DeepSORT, torchreid, DeepFace) degrades gracefully to a lightweight fallback rather than crashing when a dependency is broken. This is good defensive design but has a sharp edge — a broken dependency and a working one are indistinguishable from the pipeline's console output unless the active backend is explicitly asserted. This project's regression test suite (`tests/test_regressions.py`) exists specifically because two backends (DeepSORT, OSNet) were found silently degraded mid-project, and a third defect (inverted helmet-compliance semantics from a config/weights class-id mismatch) shipped into a published report before being caught.

### 8.3 Limitations

- Granular helmet sub-classes from the original contract taxonomy (strap-unfastened, helmet-on-handlebar, helmet-on-arm) have no available public dataset and are not modelled; the 7-class scheme used here follows the AI City Track-5 convention instead.
- Cross-camera ReID uses an off-the-shelf backbone; VeRi-776-specific fine-tuning (data already downloaded) is future work, not a current claim.
- Occlusion-voting is evaluated on UA-DETRAC, which contains no two- or three-wheelers — the vehicle classes this project otherwise focuses on — so the ablation's class coverage does not match the detection models' target domain.
- The demographics module reports gender more reliably than age given the resolution of typical face crops in this footage (median 38px against DeepFace's ~64px comfort zone for age).

### 8.4 Ethical Considerations

Automated demographic inference (age, gender) on riders raises privacy and potential-bias concerns independent of model accuracy; this system is positioned as a research prototype and any operational deployment would require explicit data-governance, consent, and bias-audit steps beyond this report's scope. Risk scoring that could inform enforcement action carries similar weight — the calibration finding in Section 6.3 (HIGH is currently unreachable from the most common real violation pattern) is flagged specifically because miscalibrated automated risk scoring has real-world consequences if deployed without human review.

### 8.5 Reproducibility Practice

Every quantitative claim in this report is generated by a script that reads exclusively from `results/phase1..4/*.json` — the same files consumed by the individual `Phase1..4_Report.pdf` documents — so no number here can silently drift from what the code actually measured. Where a metric was corrected mid-project (the wheel-classification benchmark, after the leakage fix), the superseded value and the reason for the change are recorded alongside the corrected one rather than quietly overwritten, both in the tracked JSON and in this report.

## 9. Conclusion & Future Work

---

This project delivers a complete, CPU-optimized traffic surveillance pipeline spanning adaptive preprocessing (0.79 scene-classification accuracy), a leak-free three-way detection benchmark (0.861 wheel-count accuracy, 0.764 helmet mAP@50), multi-frame tracking with a measured occlusion-robustness benefit, and model optimization that recovers accuracy lost to naive pruning while shrinking deployed models 3.8-3.9x. Every phase surfaced at least one result that did not match initial expectations; each was investigated to a root cause rather than reported around, which is the standard this report holds itself to throughout.

### Future work:

- Fine-tune OSNet on VeRi-776's training split with a triplet/ID loss to close the cross-camera ReID gap to published SOTA (issue #14).
- Source or annotate images for the granular helmet sub-classes (strap, handlebar, arm) missing from public datasets.
- Re-run the occlusion-voting ablation on footage containing two- and three-wheelers, to align the ablation's domain with the detection models it is meant to support.
- Revisit the risk-indexer weights given the real-world finding that HIGH is currently unreachable from the most common violation pattern.
- Explore augmentation-based (not duplication-based) oversampling for the passenger-helmet classes.

## 10. References

---

- [1] Wojke, N., Bewley, A., & Paulus, D. (2017). Simple Online and Realtime Tracking with a Deep Association Metric. *ICIP*.
- [2] Zhou, K., Yang, Y., Cavallaro, A., & Xiang, T. (2019). Omni-Scale Feature Learning for Person Re-Identification. *ICCV*.
- [3] Liu, X., Liu, W., Ma, H., & Fu, H. (2016). Large-Scale Vehicle Re-Identification in Urban Surveillance Videos. *ICME*. (VeRi-776)
- [4] Lyu, C., Zhang, W., Huang, H., et al. (2022). RTMDet: An Empirical Study of Designing Real-Time Object Detectors. *arXiv:2212.07784*.
- [5] Wen, L., Du, D., Cai, Z., et al. (2020). UA-DETRAC: A New Benchmark and Protocol for Multi-Object Detection and Tracking. *Computer Vision and Image Understanding*.
- [6] Jocher, G., et al. (2023). Ultralytics YOLOv8. [github.com/ultralytics/ultralytics](https://github.com/ultralytics/ultralytics).
- [7] Naik, B., et al. (2019). HELMET: A Dataset of Helmet Use in Motorcycle Traffic (Myanmar). [osf.io/4pwj8](https://osf.io/4pwj8).
- [8] AI City Challenge (2023). Track 5: Detecting Violation of Helmet Rule for Motorcyclists. [aicitychallenge.org](https://aicitychallenge.org).

# Glossary and Abbreviations

---

**mAP@50.** Mean Average Precision at 0.5 IoU threshold — the standard object-detection accuracy metric used throughout Phase 2.

**Rank-1 / Rank-5.** Re-identification accuracy: whether the correct match appears in the top-1 / top-5 ranked gallery results for a query.

**Balanced accuracy.** Macro-averaged per-class recall; used instead of raw accuracy where classes are imbalanced (Section 6.1).

**Flip-rate.** Fraction of consecutive frame-pairs (same tracked vehicle) where the emitted class label changes — a class-composition-independent stability metric.

**Accuracy retention.** Optimized-model accuracy divided by its own FP32 baseline accuracy; the headline Phase 4 metric.

**CSPNeXt.** The convolutional backbone used by RTMDet; reimplemented here in PyTorch since the official RTMDet framework does not install in this environment.

**Leak-free split.** A train/validation split constructed so that no source image contributes samples to both sides — see Section 5.1.

**Process isolation.** Running a component in a separate OS process to avoid an in-process library conflict — applied to demographics (Section 6.4).

## Appendix A — Training Hyperparameters

---

| Model                 | Epochs | Image size | Batch | Optimizer           | Notes                       |
|-----------------------|--------|------------|-------|---------------------|-----------------------------|
| Scene classifier (RF) | —      | —          | —     | 300 trees           | 15 hand-engineered features |
| SmallCNN (wheel)      | 12     | 96×96      | 32    | Adam, cosine LR     | 0.24M params, from scratch  |
| CSPNeXt (wheel)       | 12     | 96×96      | 32    | Adam, cosine LR     | 2.35M params, from scratch  |
| YOLOv8-cls (wheel)    | 8      | 96×96      | 32    | Ultralytics default | ImageNet-pretrained         |
| YOLOv8n (helmet)      | 25     | 416×416    | 8     | Ultralytics default | COCO-pretrained backbone    |
| Pruning fine-tune     | 3      | 96×96      | 32    | Adam, lr=1e-4       | post-prune recovery         |

## Appendix B — Dataset Summary

---

| Dataset              | Used for                              | Size                                        |
|----------------------|---------------------------------------|---------------------------------------------|
| DAWN                 | Fog/Rain scene classification         | 300 fog + 200 rain images                   |
| ExDark               | Night scene classification            | 300 images (of 7,363 available)             |
| COCO val2017         | Day scenes + wheel-count crops        | 300 day images; 5,000 total                 |
| Helmet (7-class)     | Helmet-compliance detection           | 368 train / 65 val / 52 test                |
| Auto-rickshaw set    | 3-wheeler crops                       | 663 images                                  |
| UA-DETRAC            | Tracking + occlusion voting           | 8 sequences, 22,480 crops                   |
| VeRi-776             | Cross-camera ReID                     | 776 vehicles, 20 cameras, 51k images        |
| HELMET (annotations) | Risk-rule validation, occupancy stats | 910 clips, 283,377 instances, 10,006 tracks |

## Appendix C — Repository & Reproduction

---

Source code, trained model configs, and all tracked results:  
<https://github.com/harsh-pandhe/traffic-surveillance-system>

Every figure and table in this report is generated by `scripts/build_final_report.py` reading directly from `results/phase1..4/*.json`; full reproduction commands are in `docs/HOW_TO_RUN.md`.

## Appendix D — Contract Deliverables Checklist

---

| Deliverable                          | Status | Evidence                                                       |
|--------------------------------------|--------|----------------------------------------------------------------|
| Complete Python source code          | Done   | This repository                                                |
| Trained model weights (PyTorch/ONNX) | Done   | <code>weights/</code> , <code>results/phase4/</code>           |
| Dataset preprocessing scripts        | Done   | <code>scripts/build_*</code> , <code>src/preprocessing/</code> |
| Installation guide + requirements    | Done   | <code>README.md</code> , <code>requirements.txt</code>         |
| Step-by-step execution documentation | Done   | <code>docs/HOW_TO_RUN.md</code>                                |
| Publication-ready research paper     | Done   | <code>docs/paper/</code>                                       |
| Per-milestone demonstration          | Done   | <code>Phase1..4_Report.pdf</code>                              |